Royaume du Maroc

Université Mohammed V de Rabat

École Mohammadia d'Ingénieurs

Département Modélisation et Informatique Scientifique (MIS)

RAPPORT DE PROJET

Module : Big Data – Cycle Ingénieur

Apache Spark · PySpark · Spark SQL · MLlib

Prédiction du niveau de mobilité quotidienne à Dakar

Exploratory Data Analysis et Machine Learning distribué avec Apache Spark

Réalisé par :

Hayat WALDI

Khaoula ELHONNYA

Encadré par :

Pr. BENHLIMA

Données :

Enquête ménages-déplacements, Dakar

Année universitaire 2026–2027

Remerciements

Nous tenons à remercier sincèrement **Pr. BENHLIMA** pour son enseignement dans le cadre du module Big Data, ainsi que pour ses conseils et sa disponibilité tout au long de ce projet. Ce travail nous a permis de mettre en pratique, sur des données réelles, les notions de traitement distribué, de Spark SQL et de Machine Learning avec Spark MLlib abordées en cours.

Nous remercions également l'**École Mohammadia d'Ingénieurs** et le **Département Modélisation et Informatique Scientifique (MIS)** pour le cadre de formation qui a rendu ce projet possible.

Résumé

Ce rapport présente un projet de Big Data mené sur l'enquête ménages-déplacements de la région de Dakar : à partir du profil d'une personne et de son ménage, nous cherchons à prédire son **niveau de mobilité quotidienne** (peu mobile, mobile ou très mobile) à l'aide d'Apache Spark et de Spark MLlib.

Les 45 605 déplacements fournis (33 variables) ont d'abord fait l'objet d'une **Exploratory Data Analysis (EDA)** avec pandas : contrôle de la qualité (heures codées au-delà de minuit, coût codé `99999`, codes de zone spéciaux), distributions, outliers, bivariate analysis et correlations. Ses conclusions ont guidé la construction d'une **chaîne de traitement distribuée** avec Spark : ingestion avec schéma explicite, nettoyage, agrégation des déplacements en 11 457 individus, variables de ménage calculées par window functions, statistiques pondérées en Spark SQL, puis modélisation avec Spark MLlib (Logistic Regression, Decision Tree, Random Forest). Les données sont découpées par ménage en **train, validation et test sets (70 / 15 / 15 %)** : le validation set sert à comparer les modèles et à régler leurs hyperparameters, et le test set n'est utilisé qu'une seule fois, pour l'évaluation finale.

L'étude montre qu'un Dakarois mobile effectue en moyenne **3,9 déplacements par jour**, que les jeunes, les élèves, les hommes et les habitants de la banlieue (Rufisque, Guédiawaye) sont les plus mobiles, et que 88 % des individus font un nombre pair de déplacements (logique d'allers-retours). Le modèle retenu, un **Random Forest**, atteint sur le test set (ménages jamais vus) une **accuracy de 44,7 %**, contre 36,1 % pour la baseline, un score très proche de celui obtenu sur le validation set (45,1 %). Ce gain modeste montre que le profil socio-démographique n'explique qu'une partie de la mobilité ; l'absence de variables comme le revenu, la profession, la motorisation ou les distances en est la principale limite.

Mots-clés : Big Data ; Apache Spark ; PySpark ; Spark SQL ; MLlib ; mobilité urbaine ; Dakar ; multiclass classification ; EDA ; data leakage ; Machine Learning.

Table des matières

Remerciements **i**

Résumé **ii**

Liste des figures **vi**

Liste des tableaux **vi**

Introduction générale **1**

1 Présentation des données **3**
1.1 Source et structure . . . 3
1.2 Variables . . . 3
1.3 Volumétrie et unité d'analyse . . . 4
1.4 Synthèse du chapitre . . . 4

2 Méthodologie et architecture **5**
2.1 Démarche générale . . . 5
2.2 Environnement technique . . . 5
2.3 Pourquoi Spark ? . . . 6
2.4 Synthèse du chapitre . . . 6

3 Exploratory Data Analysis (EDA) **7**
3.1 Qualité des données . . . 7
3.1.1 Missing values et doublons . . . 7
3.1.2 Anomalies et outliers . . . 7
3.2 Distributions des variables . . . 8
3.3 Le rythme d'une journée . . . 10
3.4 Passage au niveau individu et target . . . 10
3.5 Bivariate analysis . . . 11
3.6 Correlation matrix . . . 13
3.7 Synthèse du chapitre : du constat à la décision . . . 14

4 Préparation des données avec Apache Spark **15**
4.1 Session Spark . . . 15
4.2 Ingestion . . . 15

4.3 Contrôles de qualité . . . 15
4.4 Nettoyage . . . 16
4.5 Agrégation des déplacements en individus . . . 17
4.6 Variables du ménage par window functions . . . 18
4.7 Statistiques pondérées avec Spark SQL . . . 18
4.8 Synthèse du chapitre . . . 19

5 Modélisation avec Spark MLlib 20
5.1 Features . . . 20
5.2 Deux scénarios . . . 20
5.3 Train / validation / test split . . . 20
5.4 Préparation des features . . . 21
5.5 Modèles comparés . . . 21
5.6 Métriques d'évaluation . . . 22
5.7 Démarche de sélection en trois étapes . . . 22
5.8 Synthèse du chapitre . . . 23

6 Résultats 24
6.1 Étape 1 : comparaison des modèles sur le validation set . . . 24
6.2 Étape 2 : réglage des hyperparameters . . . 25
6.3 Étape 3 : évaluation finale sur le test set . . . 26
6.4 Confusion matrix . . . 27
6.5 Faux positifs et faux négatifs . . . 27
6.6 Feature importance . . . 28
6.7 Analyse des erreurs . . . 29
6.8 Confiance du modèle . . . 29
6.9 Exemple d'utilisation du modèle . . . 30
6.10 Synthèse du chapitre . . . 30

7 Discussion 31
7.1 Interprétation . . . 31
7.2 Limites . . . 31
7.3 Bonnes pratiques Big Data mises en œuvre . . . 32

Conclusion générale 33

A Dictionnaire des variables construites 34

B Règle de calcul du département 35

Table des figures

2.1 Chaîne de traitement du projet 5

3.1 Pourcentage de valeurs manquantes par variable 7
3.2 Boxplots de la durée, du coût et de l'âge 7
3.3 Distributions des variables numériques 9
3.4 Distribution des principales variables catégorielles 9
3.5 Nombre de départs par heure selon la famille de motifs 10
3.6 Journée d'un élève de 12 ans de l'enquête : quatre déplacements 10
3.7 Nombre de déplacements par personne, découpé en trois classes (à gauche), et part de chaque classe (à droite) 11
3.8 Nombre moyen de déplacements selon l'âge et le sexe, l'activité, la position dans le ménage et la taille du ménage 12
3.9 Composition des classes de mobilité selon l'âge, l'activité, le sexe et le département 12
3.10 Correlation matrix au niveau individu 13

4.1 Coût et heure de départ avant et après nettoyage 16
4.2 Déplacements par jour selon l'âge et le sexe (estimation pondérée) . . . 19

5.1 Taille des trois jeux et répartition des classes dans chacun 21

6.1 Accuracy sur le validation set (gris : baseline, clair : scénario A, foncé : scénario B) 24
6.2 Accuracy sur le train set et sur le validation set selon les hyperparameters 25
6.3 Accuracy du modèle final et de la baseline sur le validation set et sur le test set 26
6.4 Confusion matrix du modèle final sur le test set (effectifs et pourcentages par classe réelle) 27
6.5 TP, FP et FN par classe, et confusion matrix colorée (vert : bien classés, rouge : erreurs) 28
6.6 Feature importance du Random Forest retenu 28
6.7 Accuracy du modèle final selon le profil (test set) 29
6.8 Probabilité de la classe prédite pour les bonnes prédictions et pour les erreurs 29
6.9 Probabilités prédites pour quatre nouveaux profils 30

Liste des tableaux

1.1 Contenu du classeur fourni 3
1.2 Familles de variables de la table `deplacement` 3

2.1 Environnement logiciel 5

3.1 Anomalies détectées et traitement retenu 8
3.2 Définition de la target 11
3.3 Décisions de préparation issues de l'EDA 14

4.1 Un ménage avant et après l'agrégation (ménage n° 2) 17
4.2 Mobilité pondérée selon le sexe, l'activité et le département de résidence 19

5.1 Features du modèle 20
5.2 Rôle et taille des trois jeux 21
5.3 Modèles comparés, réglages de départ et valeurs testées 22

6.1 Performances sur le validation set (1 710 individus), réglages de départ 24
6.2 Meilleur réglage de chaque modèle (scénario B) 26
6.3 Performances sur le test set (1 660 individus de ménages jamais vus) . . 26
6.4 TP, FP, FN, precision et recall par classe (test set) 27
6.5 Prédictions pour quatre profils fictifs 30

7.1 Bonnes pratiques Big Data appliquées 32

A.1 Variables créées au niveau individu 34

Introduction générale

La région de Dakar concentre près d'un quart de la population du Sénégal et l'essentiel de son activité économique. Sa croissance rapide, portée par l'extension de la banlieue (Pikine, Guédiawaye, Rufisque), pose de forts enjeux de mobilité : accès à l'emploi, à l'école et aux services, saturation des axes vers le centre, et place prépondérante de la marche et des transports collectifs artisanaux (cars rapides, Ndiaga Ndiaye, Tata).

Les enquêtes ménages-déplacements sont l'outil de référence pour comprendre ces pratiques : elles décrivent, pour chaque membre d'un échantillon de ménages, l'ensemble des déplacements réalisés au cours d'une journée. Le *taux de mobilité*, c'est-à-dire le nombre de déplacements par personne et par jour, y est un indicateur central : il mesure l'accès effectif d'un individu aux opportunités de la ville.

Ce projet, réalisé dans le cadre du module Big Data, exploite un extrait de cette enquête. Il pose la problématique suivante :

Problématique

À partir des caractéristiques d'une personne, de la composition de son ménage et de sa zone de résidence, peut-on prédire son niveau de mobilité quotidienne, et quels facteurs la déterminent le plus ?

Contrairement à une approche par déplacement (prédire le mode, le motif ou la durée d'un trajet), ce projet se place **au niveau de l'individu**. Il faut donc transformer les données par agrégation, une opération typique des traitements Big Data. Les objectifs sont les suivants :

1. réaliser une **EDA** (Exploratory Data Analysis) complète pour évaluer la qualité des données et identifier les déterminants de la mobilité ;
2. construire une **chaîne de traitement distribuée** avec Apache Spark : ingestion, nettoyage, agrégation et ingénierie des variables ;
3. produire des **statistiques pondérées** représentatives de la population ;
4. entraîner, optimiser et évaluer des **modèles de classification** avec Spark MLlib, puis en interpréter les résultats.

Le chapitre 1 présente les données et le chapitre 2 décrit la méthodologie et l'architecture technique. Le chapitre 3 détaille l'EDA, le chapitre 4 la préparation des données avec Spark et le chapitre 5 la modélisation. Les résultats sont présentés au chapitre 6 et discutés au chapitre 7, avant la conclusion générale.

Chapitre 1

Présentation des données

1.1 Source et structure

Les données sont fournies sous la forme d'un classeur Excel (`donnees_deplacement_PSpark_2026.xlsx`) de quatre feuilles :

Tab. 1.1. Contenu du classeur fourni

Feuille	Contenu
`deplacement`	Table principale : une ligne par déplacement (45 605 lignes, 33 colonnes).
`matrice_OD`	Tableau croisé dynamique origine–destination pondéré par strate.
`explication`	Dictionnaire partiel des variables.
`strate_commune`	Correspondance entre les codes de strate et les 52 communes.

1.2 Variables

L'unité statistique de la table `deplacement` est le **déplacement**, c'est-à-dire le fait de se rendre d'une origine à une destination pour un motif donné, éventuellement en plusieurs trajets. Les 33 variables se répartissent en six familles (tableau 1.2).

Tab. 1.2. Familles de variables de la table `deplacement`

Famille	Variables
Identifiants	`numMena`, `num_individu`, `num_Deplacement`
Spatiales	`LieuDepart`, `Origine_StrateDepart`, `LieuArrivee`, `Dextination_StrateArrivee`
Temporelles	`HeureDepart`, `HeureArrivee` (format HHMM), `Duree (mn)`, `JourEnqueteIndi`
Individu	`Age`, `Sexe`, `PositMenage`
Déplacement	`Motif`, `NbTrajets`, `Mode1`…`Mode4`, `NbTrajets*` (10 colonnes), `Cout`, `Moyen de transport`
Pondération	`Poids_extra` (poids d'extrapolation)

1.3 Volumétrie et unité d'analyse

Le fichier décrit **45 605 déplacements** réalisés par **11 457 individus** de **3 127 ménages**, âgés de 11 à 95 ans. La variable `Poids_extra` indique combien de déplacements réels chaque ligne représente : sa somme estime à environ **6,7 millions** le nombre de déplacements quotidiens dans la population.

Notre question portant sur les personnes, l'unité d'analyse du modèle est l'**individu** : les données doivent donc être agrégées (chapitre 4).

1.4 Synthèse du chapitre

Le jeu de données décrit 45 605 déplacements, réalisés par 11 457 individus de 3 127 ménages et décrits par 33 variables. L'unité statistique d'origine est le déplacement ; notre question portant sur les personnes, les données devront être agrégées au niveau de l'individu.

Chapitre 2

Méthodologie et architecture

2.1 Démarche générale

Le projet suit une démarche classique de science des données, adaptée au contexte Big Data : l'exploration se fait sur un échantillon en mémoire (pandas), tandis que la préparation et la modélisation sont implémentées avec Spark, de façon à pouvoir passer à l'échelle sans modification du code.

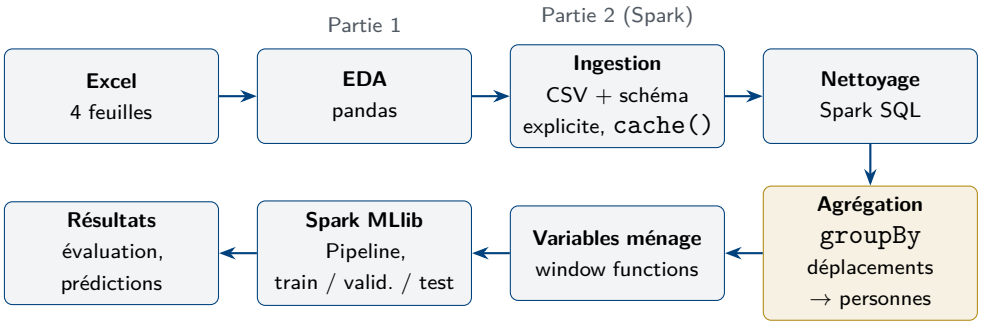


Fig. 2.1. Chaîne de traitement du projet

2.2 Environnement technique

Tab. 2.1. Environnement logiciel

Composant	Version / configuration
Apache Spark (PySpark)	3.5.3, mode `local[*]`
Système	Windows (Anaconda, JupyterLab)
Java	OpenJDK 17 (Eclipse Temurin)
Python	3 (Anaconda) avec pandas, NumPy, Matplotlib, Seaborn
Partitions de shuffle	`spark.sql.shuffle.partitions = 8`
Mémoire du driver	4 Go
Stockage des tables	En mémoire avec `cache()`

Exécution sous Windows

Le notebook a été exécuté sous Windows. Pour que Spark fonctionne, nous avons configuré Java (`JAVA_HOME`), conservé les tables intermédiaires en mémoire avec `cache()` au lieu de les écrire sur le disque, et utilisé uniquement des fonctions Spark natives. Les modèles sont entraînés l'un après l'autre pour respecter la mémoire disponible.

2.3 Pourquoi Spark ?

Le jeu de données tient en mémoire, mais une enquête de ce type peut être répétée sur plusieurs années, étendue à d'autres villes ou enrichie de traces GPS. Les choix techniques visent donc la **scalabilité** :

- **mise en cache** (`cache()`) : les tables réutilisées restent en mémoire et ne sont pas recalculées à chaque étape ;
- **transformations distribuées** (`groupBy`, fenêtres, jointures) : aucune donnée n'est rapatriée sur le driver, sauf les petits résultats agrégés destinés aux graphiques ;
- **MLlib** : pipelines reproductibles (préparation + modèle), entraînés de façon distribuée.

2.4 Synthèse du chapitre

Le projet combine une exploration en mémoire avec pandas et une chaîne de traitement Spark conçue pour passer à l'échelle.

Chapitre 3

Exploratory Data Analysis (EDA)

L'EDA est menée avec pandas, d'abord au niveau des déplacements (qualité des données), puis au niveau des personnes (target et déterminants).

3.1 Qualité des données

3.1.1 Missing values et doublons

Seules cinq variables présentent des valeurs manquantes (figure 3.1). Les colonnes `Mode2` à `Mode4` ne sont renseignées que lorsque le déplacement compte plusieurs trajets : leur vacuité est **structurelle** et ne doit pas être imputée. `Cout` (11 lignes) et `NbTrajetsAPied` (1 ligne) sont marginales. Aucun doublon n'existe sur la clé (ménage, individu, numéro de déplacement).

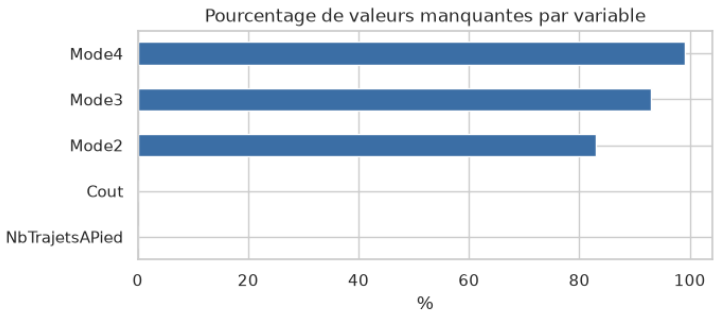


Fig. 3.1. Pourcentage de valeurs manquantes par variable

3.1.2 Anomalies et outliers

Les statistiques descriptives et les boxplots (figure 3.2) révèlent plusieurs anomalies, résumées dans le tableau 3.1.

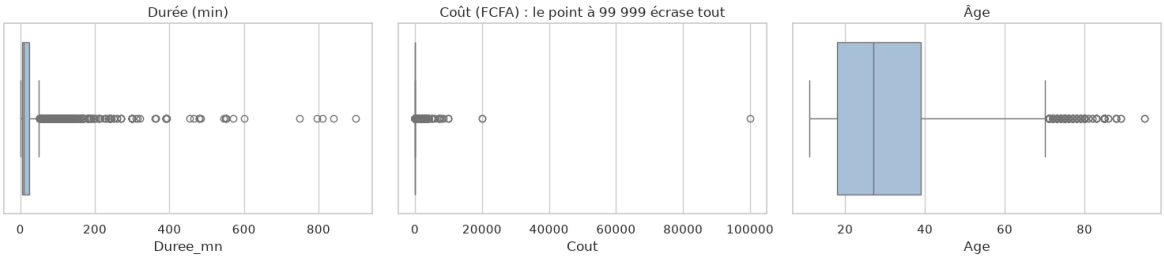


Fig. 3.2. Boxplots de la durée, du coût et de l'âge

Tab. 3.1. Anomalies détectées et traitement retenu

Anomalie	**Lignes**	**Traitement**
Coût codé `99999` (« inconnu »)	1	remplacé par `null`
Heure de départ $\geq$ 2400 (après minuit, format HHMM)	270	heure ramenée sur 0–23 h pour l'analyse
Durée supérieure à 3 heures	83	conservée (compte comme déplacement)
Codes de strate spéciaux (`zex`, `NA_d`, `NA_p`…)	519	règle dédiée pour le département
Durée incohérente avec les heures (écart $>$ 5 min)	0	aucun

Point d'attention

La règle de l'IQR (InterQuartile Range) signale plus de 3 000 « outliers » pour la durée et plus de 10 000 pour le coût. Il s'agit en réalité de distributions très **asymétriques** (marche gratuite et courte contre transports collectifs longs et payants), et non d'erreurs. Seules les anomalies du tableau 3.1 sont traitées.

3.2 Distributions des variables

La figure 3.3 présente les distributions des principales variables numériques. La population enquêtée est jeune (âge médian de 27 ans). Les durées sont courtes (médiane de 10 minutes) avec une longue traîne. Les départs se concentrent autour de trois pics (7 h, 10 h et 17–19 h). 77 % des déplacements sont gratuits, et 83 % se font en un seul trajet.

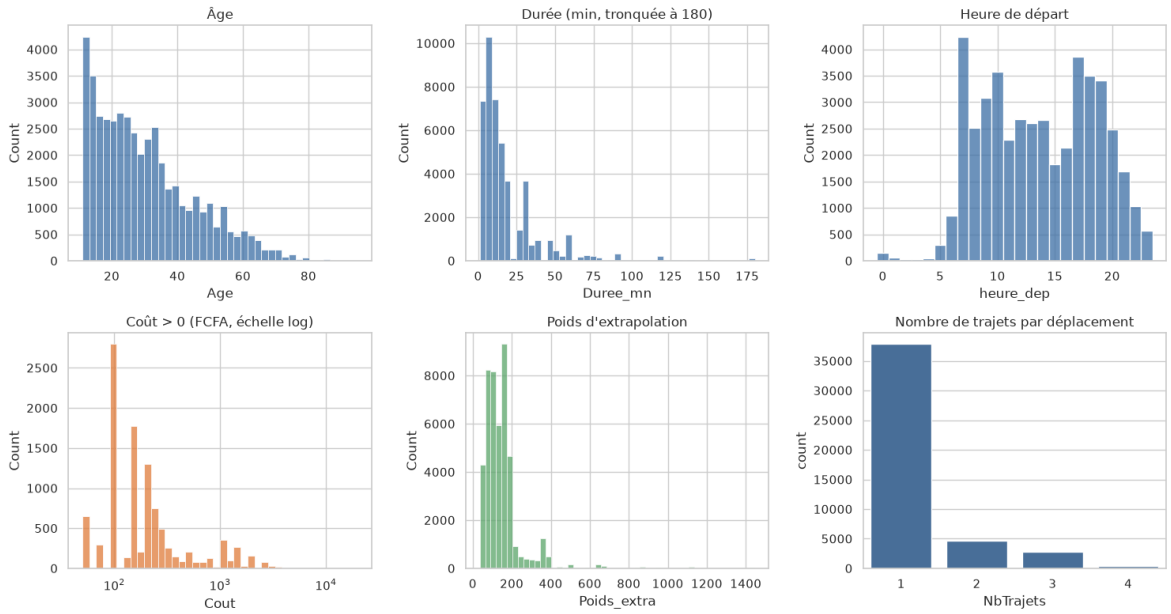


Fig. 3.3. Distributions des variables numériques

Les variables catégorielles (figure 3.4) confirment la prédominance de la marche (70 % des déplacements) et du motif « retour au domicile » (43 %). L'échantillon est équilibré entre hommes (50,7 %) et femmes (49,3 %) ; le lundi n'est pas enquêté.

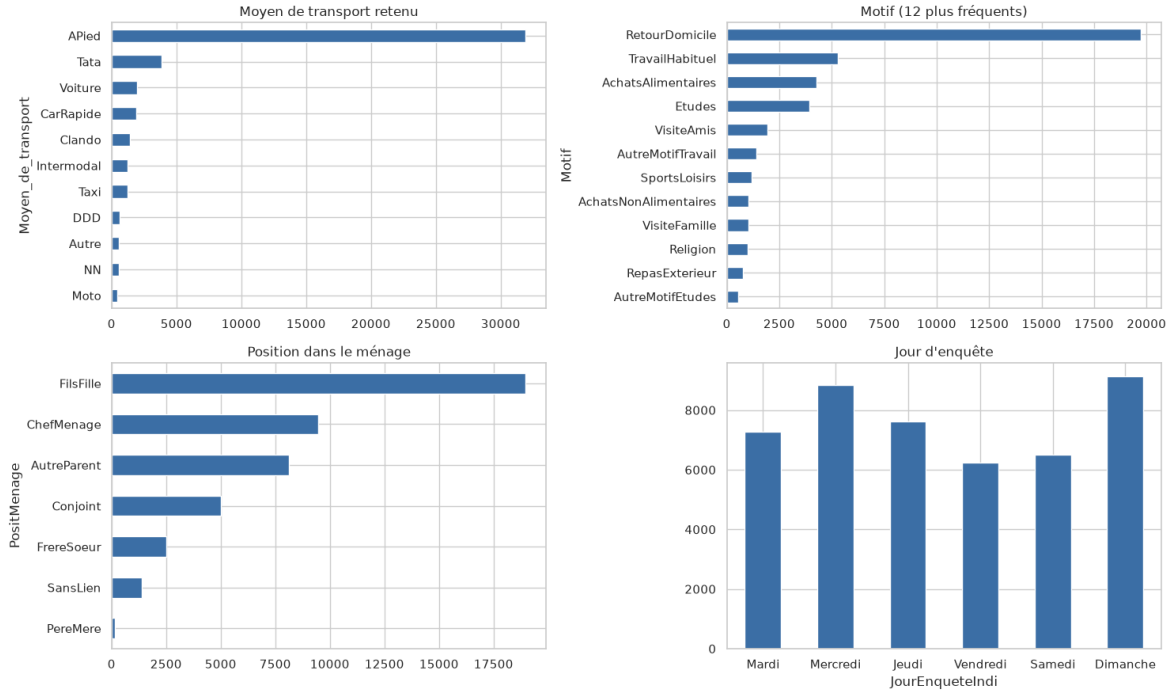


Fig. 3.4. Distribution des principales variables catégorielles

3.3 Le rythme d'une journée

Pour comprendre concrètement ce qu'est un déplacement, la figure 3.5 montre à quelle heure on part selon le motif, et la figure 3.6 reconstitue la journée réelle d'un élève de l'enquête.

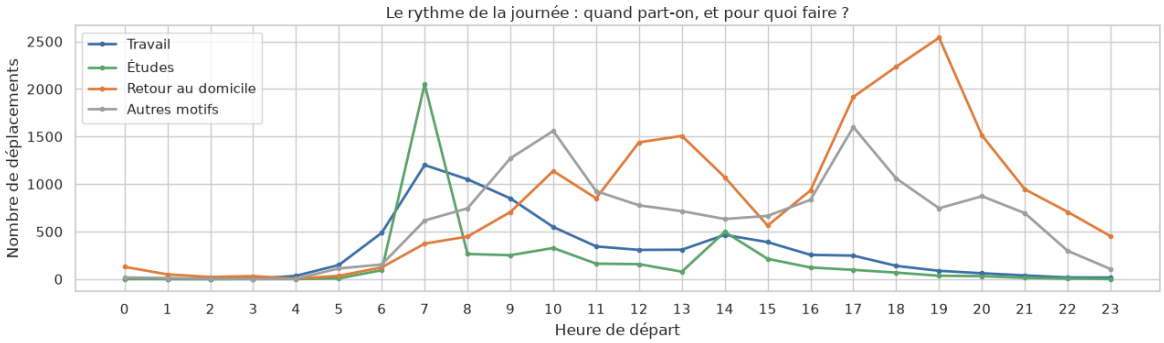


Fig. 3.5. Nombre de départs par heure selon la famille de motifs

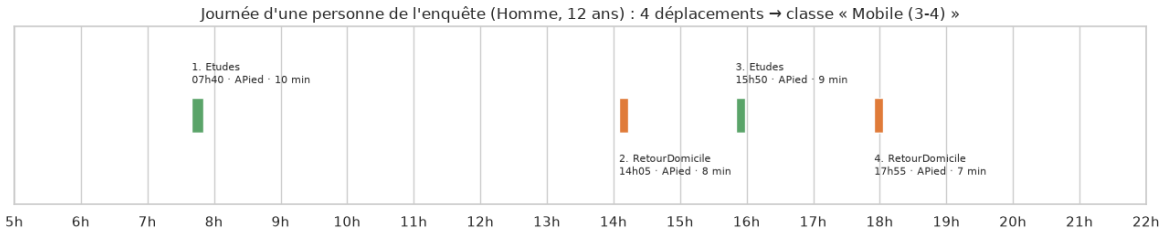


Fig. 3.6. Journée d'un élève de 12 ans de l'enquête : quatre déplacements

Les départs pour les études et le travail culminent à 7 h ; les retours au domicile forment deux pics, vers 12–13 h (retour pour le déjeuner) et vers 19 h. La journée de l'élève suit la logique des **allers-retours** : domicile → école → domicile, deux fois dans la journée, soit 4 déplacements. Le dernier déplacement ramène la personne chez elle, ce qui permettra de connaître sa zone de résidence.

3.4 Passage au niveau individu et target

Les déplacements sont regroupés par individu, et l'on compte leur nombre dans la journée. Le nombre moyen est de **4 déplacements** par personne (médiane 4, maximum 18).

La figure 3.7 met en évidence un phénomène remarquable : **88 % des individus effectuent un nombre pair de déplacements**, ce qui traduit la logique des allers-retours depuis le domicile. De plus, 98 % des journées se terminent par un retour au domicile, ce qui permet de déduire la **zone de résidence** comme la strate d'arrivée du dernier déplacement.

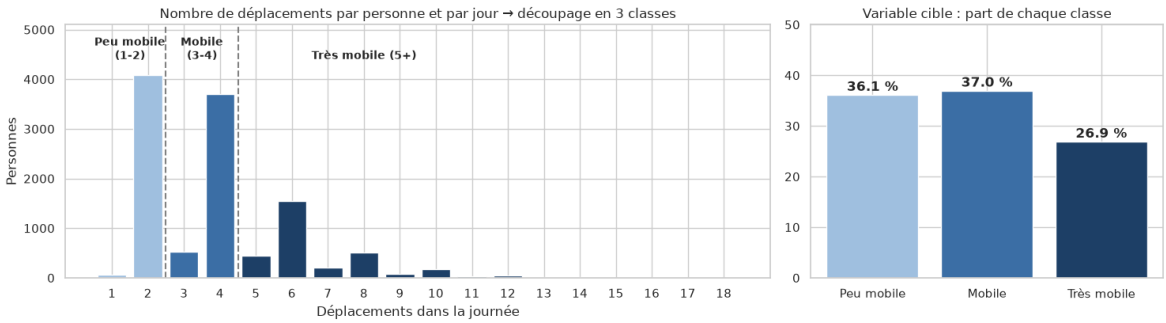


Fig. 3.7. Nombre de déplacements par personne, découpé en trois classes (à gauche), et part de chaque classe (à droite)

La target est définie en trois classes, dont les effectifs sont équilibrés :

Tab. 3.2. Définition de la target

Code	Niveau de mobilité	Déplacements / jour	Part des individus
0	Peu mobile	1 – 2	36,1 %
1	Mobile	3 – 4	37,0 %
2	Très mobile	5 et plus	26,9 %

Limite de la target

L'enquête ne contient que des personnes s'étant déplacées au moins une fois. Le modèle prédit donc l'**intensité** de la mobilité, et non le fait de se déplacer ou non.

3.5 Bivariate analysis

La figure 3.8 croise le nombre moyen de déplacements avec les principales caractéristiques des individus.

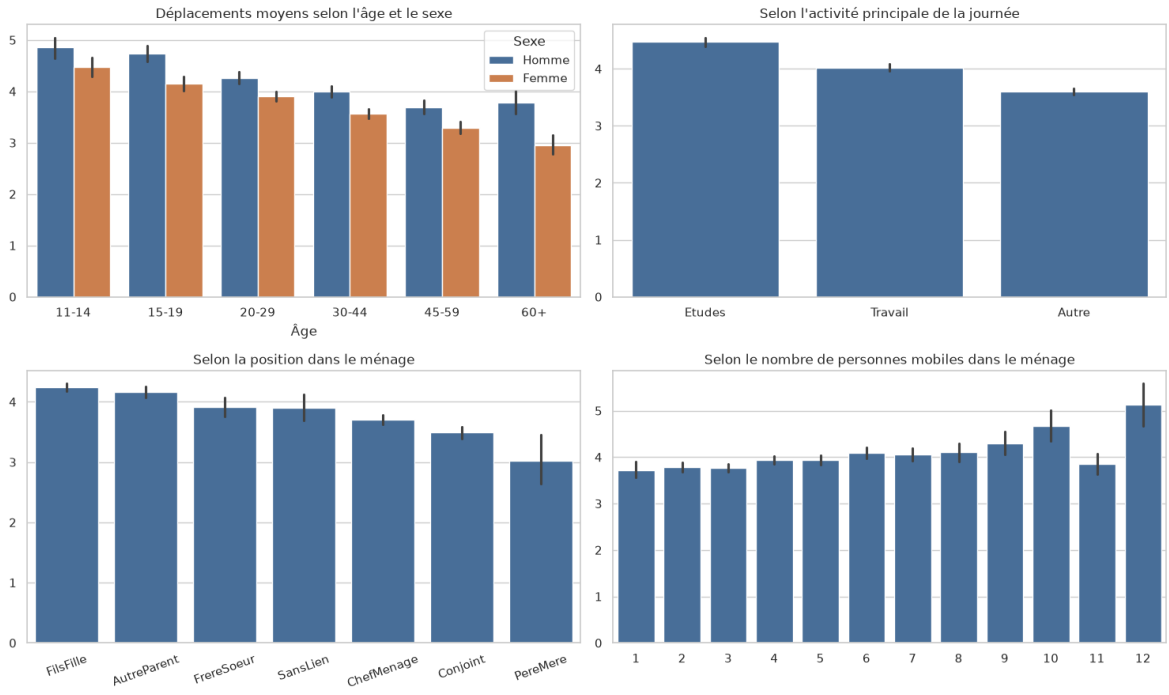


Fig. 3.8. Nombre moyen de déplacements selon l'âge et le sexe, l'activité, la position dans le ménage et la taille du ménage

Déterminants de la mobilité

- **Âge** : facteur le plus net. Les 11–14 ans font 4,65 déplacements par jour contre 3,46 après 60 ans (allers-retours scolaires, souvent matin et après-midi).
- **Activité** : élèves et étudiants (4,47) > actifs (4,02) > autres (3,60).
- **Sexe** : hommes 4,20 contre femmes 3,78.
- **Ménage** : la mobilité individuelle croît avec le nombre de personnes mobiles du ménage (de 3,7 à 4,7).
- **Jour** : effet quasi nul (3,9 à 4,1).

Les moyennes cachent la répartition. La figure 3.9 montre, pour chaque groupe, la part de peu mobiles, de mobiles et de très mobiles.

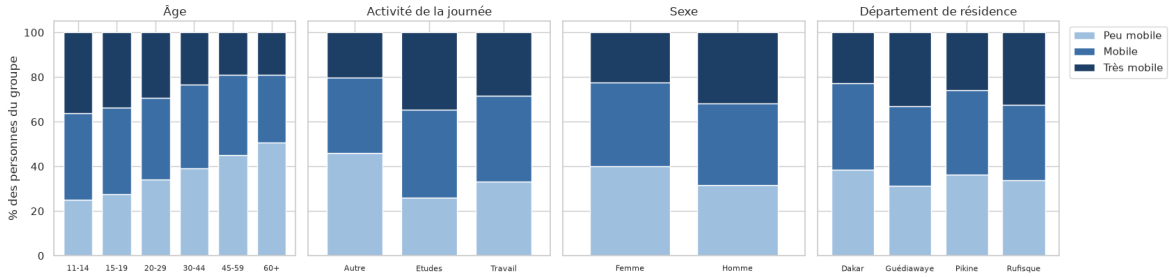


Fig. 3.9. Composition des classes de mobilité selon l'âge, l'activité, le sexe et le département

La part de très mobiles baisse avec l'âge et est la plus forte chez les élèves ; la part de peu mobiles est la plus forte après 60 ans et chez les personnes sans travail ni études ce jour-là. Mais **dans chaque groupe, les trois classes sont présentes** : aucun profil n'est « toujours très mobile », ce qui annonce une prédiction difficile.

3.6 Correlation matrix

Aucune variable n'est fortement corrélée, prise seule, au nombre de déplacements ($|r| < 0{,}2$, figure 3.10). La mobilité résulte d'une **combinaison** de facteurs, ce qui justifie de comparer plusieurs modèles multivariés, linéaires (Logistic Regression) et non linéaires (Decision Tree, Random Forest). La corrélation ne mesure toutefois que les liens linéaires entre variables numériques : les variables catégorielles, comme la zone de résidence, n'y apparaissent pas.

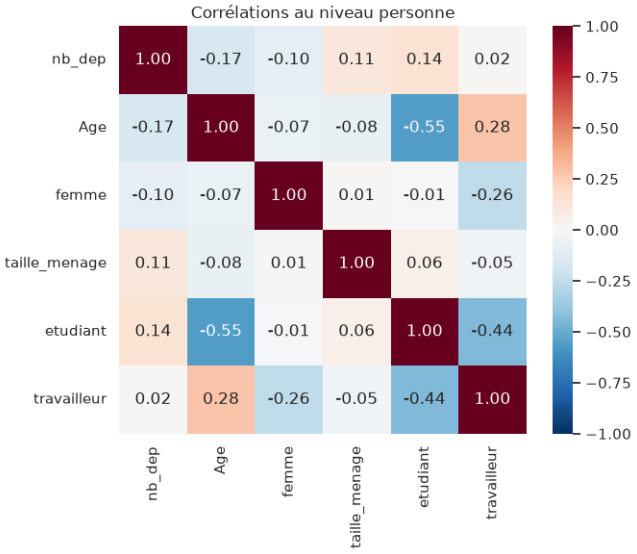


Fig. 3.10. Correlation matrix au niveau individu

Data leakage

On obtiendrait des corrélations bien plus fortes avec des variables calculées sur la journée (nombre de motifs différents, durée totale, coût total, modes utilisés). Mais elles sont calculées à partir des **mêmes déplacements** que la target : elles augmentent mécaniquement avec le nombre de déplacements et contiennent donc déjà la réponse. Elles sont **exclues** des features.

3.7 Synthèse du chapitre : du constat à la décision

Tab. 3.3. Décisions de préparation issues de l'EDA

Constat	**Décision appliquée dans Spark**
Heures HHMM, coût `99999`, durées > 3 h	Coût codé $\rightarrow$ `null`, heures converties en minutes ; déplacements longs conservés
La question porte sur les individus	Agrégation `groupBy(numMena, num_individu)`
98 % des journées finissent au domicile	Résidence = strate d'arrivée du dernier déplacement
Classes équilibrées (36 / 37 / 27 %)	Target en 3 classes, accuracy et F1-score comme métriques
Influence de la composition du ménage	Variables de ménage par window functions
Activité fortement liée à la mobilité	Deux scénarios : profil seul (A) et profil + activité (B)
Individus d'un même ménage corrélés	Split train / validation / test (70 / 15 / 15) **par ménage**
`Poids_extra` constant par individu	Statistiques pondérées à l'échelle de la population

Chapitre 4

Préparation des données avec Apache Spark

4.1 Session Spark

La session Spark est le point d'entrée de tout le traitement. Elle est lancée en mode local sur tous les cœurs du processeur (`local[*]`), avec 8 partitions après les regroupements et 4 Go de mémoire.

Extrait de code 4.1. Création de la session Spark

```
spark = (SparkSession.builder
         .appName("Mobilite_Quotidienne_Dakar")
         .master("local[*]")
         .config("spark.sql.shuffle.partitions", "8")
         .config("spark.driver.memory", "4g")
         .getOrCreate())
```

4.2 Ingestion

Spark ne disposant pas de lecteur natif pour le format `.xlsx`, la feuille `deplacement` est convertie une seule fois en CSV, puis lue par Spark avec un **schéma explicite** (33 champs typés). Un schéma explicite évite le double parcours des données qu'impose `inferSchema` et garantit les types. La table est ensuite mise en cache pour ne pas relire le fichier à chaque calcul.

Extrait de code 4.2. Lecture du CSV avec un schéma explicite

```
dep_pd = pd.read_excel(file, sheet_name="deplacement")
dep_pd.columns = [c.replace(" (mn)", "_mn").replace(" ", "_") for c in dep_pd.
    columns]
dep_pd.to_csv("data/raw/deplacement.csv", index=False)

df = spark.read.csv("data/raw/deplacement.csv", header=True, schema=schema).cache()
```

4.3 Contrôles de qualité

Les contrôles de l'EDA sont reproduits en Spark, de façon distribuée : absence de doublons, 1 coût codé, 270 départs après minuit et 83 déplacements de plus de 3 heures. On vérifie en outre que l'âge, le sexe, le jour et le poids sont **constants pour chaque**

individu, condition nécessaire pour agréger sans perte d'information.

4.4 Nettoyage

Le nettoyage applique les décisions de l'EDA : le coût codé `99999` est remplacé par une valeur vide, les heures de départ au format HHMM sont converties en minutes depuis minuit et ramenées sur 0–24 h, et deux indicateurs (0 ou 1) repèrent les déplacements effectués pour le travail et pour les études. Une fonction calcule aussi le département à partir du code de zone, en traitant les codes spéciaux (voir l'annexe).

Extrait de code 4.3. Nettoyage des déplacements

```
clean = (df
    .withColumn("Cout", F.when(F.col("Cout") >= 99999, None).otherwise(F.col("Cout")
    ))
    .withColumn("heure_dep_min",
                ((F.col("HeureDepart") / 100).cast("int") * 60 + F.col("HeureDepart"
    ) % 100) % 1440)
    .withColumn("est_travail", F.col("Motif").isin(motifs_travail).cast("int"))
    .withColumn("est_etudes", F.col("Motif").isin(motifs_etudes).cast("int"))
).cache()
```

La figure 4.1 compare les données brutes et nettoyées.

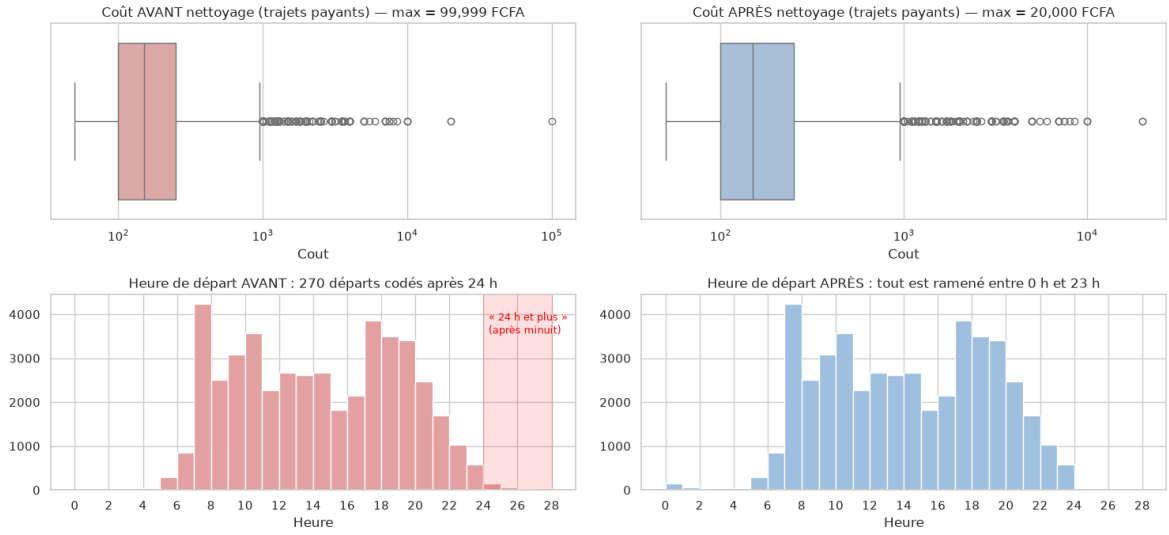


Fig. 4.1. Coût et heure de départ avant et après nettoyage

Avant nettoyage, le code `99999` étirait toute la distribution du coût ; après, le maximum est de 20 000 FCFA, un coût plausible pour un long trajet en taxi. Les 270 départs codés après « 24 h » sont ramenés au petit matin. Le nettoyage est **léger** : il ne supprime aucune ligne et ne modifie pas la target.

4.5 Agrégation des déplacements en individus

C'est l'étape clé du projet. Un `groupBy` distribué transforme les 45 605 déplacements en 11 457 individus. La fonction `max_by` récupère la strate d'arrivée du dernier déplacement (zone de résidence), et des indicateurs de motif déterminent l'activité principale de la journée.

Extrait de code 4.4. Agrégation distribuée au niveau individu

```
personnes = (clean.groupBy("numMena", "num_individu")
    .agg(F.count("*").alias("nb_dep"),
         F.first("Age").alias("Age"), F.first("Sexe").alias("Sexe"),
         F.first("PositMenage").alias("PositMenage"),
         F.first("JourEnqueteIndi").alias("Jour"),
         F.first("Poids_extra").alias("poids"),
         F.max_by("Dextination_StrateArrivee", "num_Deplacement")
          .alias("strate_residence"),
         F.max("est_travail").alias("va_travail"),
         F.max("est_etudes").alias("va_etudes"))
    .withColumn("activite", F.when(F.col("va_travail") == 1, "Travail")
                            .when(F.col("va_etudes") == 1, "Etudes")
                            .otherwise("Autre")))
```

Le tableau 4.1 illustre cette transformation sur un ménage réel de trois personnes : ses 12 déplacements deviennent 3 lignes, une par personne.

Tab. 4.1. Un ménage avant et après l'agrégation (ménage n° 2)

Avant : déplacements					**Après : personnes**		
Indiv.	**N°**	**Heure**	**Motif**	**Arrivée**	**Indiv.**	`nb_dep`	**Activité**
1	1	9h15	TravailHabituel	D01	1	4	Travail
1	2	10h30	RepasExterieur	D01			
1	3	18h00	TravailHabituel	D01			
1	4	19h15	RetourDomicile	D01			
2	1	8h00	AutreMotifTravail	D17	2	4	Travail
2	2	13h30	Religion	D02			
2	3	15h10	VisiteAmis	D04			
2	4	19h00	RetourDomicile	D01			
3	1	8h00	TravailHabituel	D04	3	4	Travail
3	2	16h15	Associations	D06			
3	3	16h30	AutreMotifMenage	D04			
3	4	19h20	RetourDomicile	D01			

4.6 Variables du ménage par window functions

Une fenêtre partitionnée par ménage calcule des statistiques collectives et les recopie sur chaque membre, sans jointure ni collecte sur le driver. Contrairement à un `groupBy`, le nombre de lignes ne change pas : dans le ménage n° 2, les trois membres gardent leur ligne et reçoivent les mêmes valeurs de ménage (taille 3, âge moyen 28,3 ans, aucune femme).

Extrait de code 4.5. Variables de ménage calculées par window functions

```
w_menage = Window.partitionBy("numMena")
personnes = (personnes
    .withColumn("taille_menage", F.count("*").over(w_menage))
    .withColumn("nb_enfants", F.sum((F.col("Age") < 18).cast("int")).over(w_menage))
    .withColumn("nb_seniors", F.sum((F.col("Age") >= 60).cast("int")).over(w_menage)
    )
    .withColumn("nb_adultes", F.col("taille_menage") - F.col("nb_enfants") - F.col("
    nb_seniors"))
    .withColumn("age_moyen_menage", F.round(F.avg("Age").over(w_menage), 1))
    .withColumn("part_femmes_menage", F.round(F.avg((F.col("Sexe") == "Femme").cast(
    "int")).over(w_menage), 2))
    .withColumn("sexe_chef", F.coalesce(
        F.max(F.when(F.col("PositMenage") == "ChefMenage", F.col("Sexe"))).over(
    w_menage), F.lit("Inconnu")))
    .withColumn("weekend", F.col("Jour").isin("Samedi", "Dimanche").cast("int"))
    .withColumn("niveau", F.when(F.col("nb_dep") <= 2, 0).when(F.col("nb_dep") <= 4,
     1).otherwise(2).cast("double"))
)
personnes = personnes.cache()
```

4.7 Statistiques pondérées avec Spark SQL

Le poids d'extrapolation permet d'estimer la mobilité de l'ensemble de la population. La population mobile de 11 ans et plus est estimée à **1,70 million** de personnes, qui réalisent **6,70 millions** de déplacements par jour, soit **3,93 déplacements** par personne.

Tab. 4.2. Mobilité pondérée selon le sexe, l'activité et le département de résidence

Variable	Modalité	Effectif	Dépl./jour	% peu mobiles	% très mobiles
Sexe	Homme	5 514	4,16	32,1	31,2
	Femme	5 943	3,72	41,4	21,5
Activité	Études	2 950	4,41	26,6	33,4
	Travail	4 231	3,99	33,7	27,9
	Autre	4 276	3,55	47,0	19,6
Département	Rufisque	2 430	4,29	35,2	32,3
	Guédiawaye	1 156	4,23	30,8	31,9
	Pikine	3 742	3,93	37,4	25,7
	Dakar	4 117	3,73	38,4	23,1

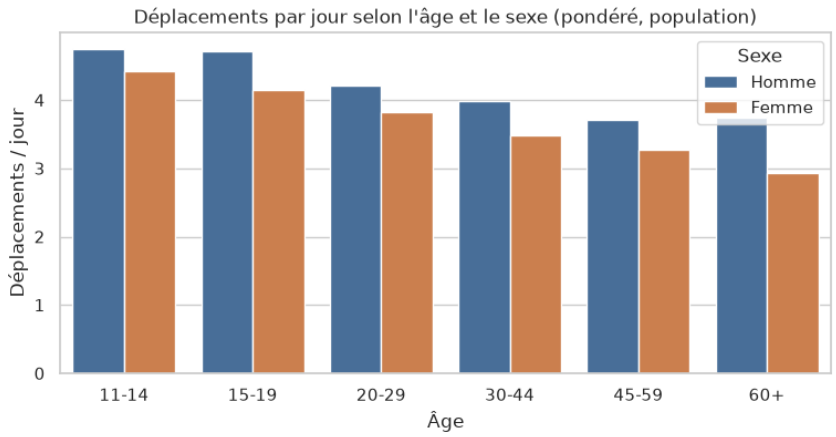


Fig. 4.2. Déplacements par jour selon l'âge et le sexe (estimation pondérée)

L'écart entre hommes et femmes existe à tout âge et s'accentue avec l'âge : il est le plus marqué après 60 ans (environ 3,7 déplacements par jour pour les hommes contre 2,9 pour les femmes), puis chez les 15–19 ans. Enfin, près d'une personne sur deux sans activité de travail ou d'études dans la journée (47 %) est peu mobile, contre 27 % des élèves et étudiants.

4.8 Synthèse du chapitre

Spark transforme les 45 605 déplacements en 11 457 individus enrichis de variables de ménage, sans rapatrier les données sur le driver. Les estimations pondérées confirment les écarts observés pendant l'exploration : les jeunes, les élèves, les hommes et les habitants de la banlieue sont les plus mobiles.

Chapitre 5

Modélisation avec Spark MLlib

5.1 Features

Tab. 5.1. Features du modèle

Type	Variables	Encodage
Catégorielles	Sexe, position dans le ménage, jour, strate et département de résidence, sexe du chef de ménage	`StringIndexer` (+ `OneHotEncoder`)
Numériques	Âge, taille du ménage, nombres d'enfants, d'adultes et de seniors, âge moyen du ménage, part de femmes, week-end	valeurs brutes (standardisées pour la Logistic Regression)
Scénario B	Activité principale de la journée (travail, études, autre)	`StringIndexer`

5.2 Deux scénarios

- **Scénario A – profil seul** : uniquement des informations connues avant la journée.
- **Scénario B – profil + activité** : ajout de l'activité principale, qui sert d'indicateur du statut de la personne (élève, actif). Cette variable n'étant pas déclarée dans l'enquête, elle est déduite des motifs de la journée ; le scénario B constitue donc une borne optimiste.

5.3 Train / validation / test split

Les membres d'un même ménage partagent des habitudes et les mêmes variables de ménage. Si une personne était dans un jeu et un membre de sa famille dans un autre, le modèle aurait déjà « vu » ce ménage et son score serait trop optimiste (data leakage). Le découpage se fait donc **par ménage**, à partir d'un hash de l'identifiant du ménage ramené entre 0 et 99 : tous les membres d'un ménage tombent dans le même jeu.

Tab. 5.2. Rôle et taille des trois jeux

Jeu	Buckets	Personnes	Ménages	Rôle
Train set	0–69	8 087 (70,6 %)	2 224	entraîner les modèles
Validation set	70–84	1 710 (14,9 %)	449	comparer les modèles, choisir les hyperparameters
Test set	85–99	1 660 (14,5 %)	454	évaluer une seule fois le modèle final

Trois jeux sont nécessaires : si le meilleur modèle était choisi en regardant le test set, il finirait par s'y adapter et son score serait trop optimiste. Le validation set sert à **faire les choix** ; le test set reste **intact** jusqu'à la fin. La figure 5.1 vérifie que les trois jeux ont des répartitions de classes très proches.

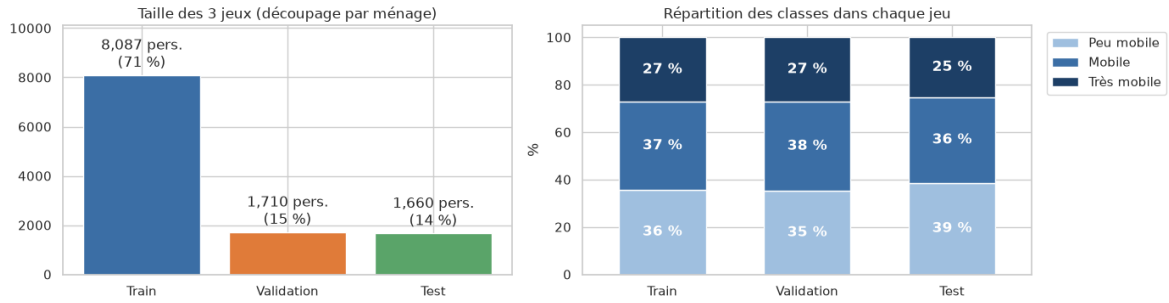


Fig. 5.1. Taille des trois jeux et répartition des classes dans chacun

5.4 Préparation des features

Les modèles de Spark MLlib attendent une seule colonne contenant un vecteur de nombres. La préparation enchaîne :

- `StringIndexer`, qui transforme chaque variable texte en numéro ;
- `OneHotEncoder` (Logistic Regression uniquement), qui transforme ces numéros en colonnes 0/1 ;
- `VectorAssembler`, qui rassemble toutes les variables dans un vecteur `features` ;
- `StandardScaler` (Logistic Regression uniquement), qui met les variables à la même échelle.

5.5 Modèles comparés

Chaque modèle est placé dans un `Pipeline` MLlib avec les étapes de préparation, entraîné sur le train set (`fit`) et évalué sur le validation set (`transform`).

Tab. 5.3. Modèles comparés, réglages de départ et valeurs testées

Modèle	Réglages de départ (étape 1)	Valeurs testées (étape 2)
Baseline	Prédit toujours la classe la plus fréquente du train set (*mobile*)	–
Logistic Regression	Multinomiale, 200 itérations, `regParam` = 0,01	`regParam` : 0,001 ; 0,01 ; 0,1 ; 0,3 ; 1
Decision Tree	`maxDepth` = 6	`maxDepth` : 3 ; 5 ; 7 ; 9 ; 12
Random Forest	100 arbres, `maxDepth` = 10	`maxDepth` : 6 ; 8 ; 10 × `minInstancesPerNode` : 1 ; 5

5.6 Métriques d'évaluation

- **Accuracy** : part des personnes correctement classées.
- **Recall par classe** : part des personnes d'une classe que le modèle retrouve (par exemple, la part des très mobiles reconnus comme très mobiles).
- **Weighted F1-score** : moyenne de la precision et du recall de chaque classe, pondérée par la taille des classes.

5.7 Démarche de sélection en trois étapes

1. **Comparer** les trois modèles dans les deux scénarios sur le validation set, avec leurs réglages de départ.
2. **Régler les hyperparameters** de chaque modèle (scénario B) : chaque réglage est entraîné sur le train set et évalué sur le train set et sur le validation set ; on garde la meilleure accuracy de validation.
3. **Évaluer une seule fois** le modèle retenu sur le test set.

Extrait de code 5.1. Réglage des hyperparameters sur le validation set

```
grilles = {
    "Logistic Regression": [{"regParam": r} for r in [0.001, 0.01, 0.1, 0.3, 1.0]],
    "Decision Tree":       [{"maxDepth": d} for d in [3, 5, 7, 9, 12]],
    "Random Forest":       [{"maxDepth": d, "minInstancesPerNode": m}
                            for d in [6, 8, 10] for m in [1, 5]],
}
for nom, grille in grilles.items():
    for params in grille:
        modele = construire(nom, cat_B, **params).fit(train)
        acc_train = mc("accuracy").evaluate(modele.transform(train))
        acc_val = mc("accuracy").evaluate(modele.transform(valid))
```

Pourquoi pas de cross-validation ?

Le `CrossValidator` de Spark découpe le train set **personne par personne** : des membres d'un même ménage se retrouvent dans des folds différents, ce qui rend le score trop optimiste. Le validation set, lui, est découpé **par ménage**, comme le test set.

5.8 Synthèse du chapitre

Trois modèles sont comparés à une baseline naïve, dans deux scénarios de variables. Les choix (scénario, modèle, hyperparameters) sont faits sur le validation set, et le test set ne sert qu'à l'évaluation finale. Le découpage par ménage empêche le data leakage entre les trois jeux.

Chapitre 6

Résultats

6.1 Étape 1 : comparaison des modèles sur le validation set

Tab. 6.1. Performances sur le validation set (1 710 individus), réglages de départ

Scénario	Modèle	Accuracy	Weighted F1	Recall		
				Peu	Mobile	Très
–	Baseline (classe majoritaire)	0,378	0,207	0,000	1,000	0,000
A	Logistic Regression	0,424	0,421	0,518	0,387	0,353
	Decision Tree	0,418	0,412	0,429	0,514	0,268
	Random Forest	0,423	0,419	0,512	0,429	0,299
B	Logistic Regression	0,441	0,438	0,542	0,410	0,353
	Decision Tree	0,425	0,388	0,508	0,594	0,078
	Random Forest	**0,445**	**0,437**	0,525	0,500	0,264

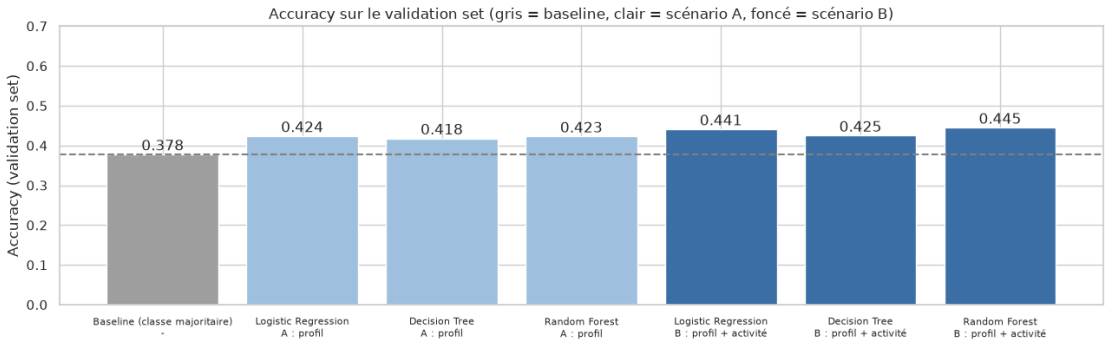


Fig. 6.1. Accuracy sur le validation set (gris : baseline, clair : scénario A, foncé : scénario B)

Lecture des résultats

- Les six modèles battent la baseline (0,378).
- **Le scénario B fait mieux que le scénario A pour les trois modèles** : l'activité de la journée aide (Random Forest 0,423 → 0,445 ; Logistic Regression 0,424 → 0,441). On garde donc le scénario B.
- Les écarts entre modèles restent faibles (0,42 à 0,45). Le Decision Tree du scénario B ne reconnaît presque aucun très mobile (recall 0,078) : un seul arbre est instable.

6.2 Étape 2 : réglage des hyperparameters

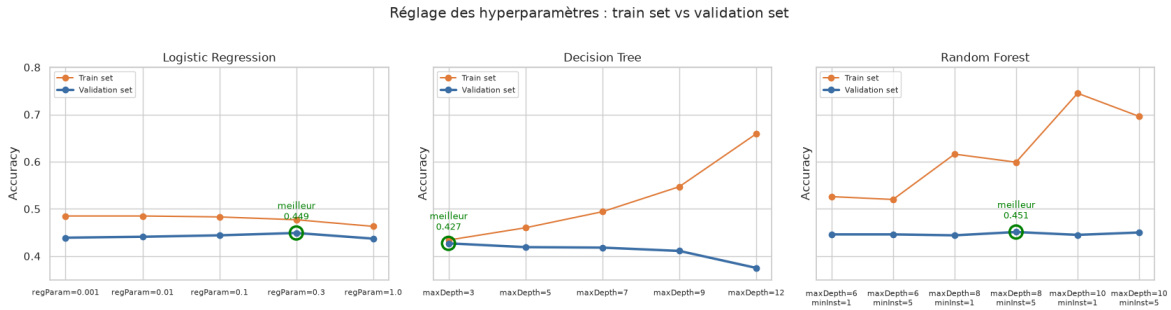


Fig. 6.2. Accuracy sur le train set et sur le validation set selon les hyperparameters

La figure 6.2 met en évidence l'**overfitting** :

- **Decision Tree** : plus l'arbre est profond, plus l'accuracy train monte (0,43 → 0,66), mais l'accuracy de validation **baisse** (0,427 → 0,375). L'arbre apprend par cœur les personnes du train set ; le meilleur réglage est le plus simple (`maxDepth` = 3).
- **Random Forest** : l'accuracy train monte aussi (0,52 → 0,75), mais l'accuracy de validation reste stable autour de 0,45. En faisant voter 100 arbres différents, la forêt **résiste** à l'overfitting. Le meilleur réglage est `maxDepth` = 8, `minInstancesPerNode` = 5.
- **Logistic Regression** : train et validation restent proches (environ 0,48 et 0,44) ; un modèle linéaire a peu de risque d'overfitting. Une régularisation modérée (`regParam` = 0,3) aide un peu ; trop forte (1), elle dégrade le modèle.

Tab. 6.2. Meilleur réglage de chaque modèle (scénario B)

Modèle	Meilleur réglage	Accuracy train	Accuracy validation
Random Forest	`maxDepth` = 8, `minInstancesPerNode` = 5	0,599	**0,451**
Logistic Regression	`regParam` = 0,3	0,477	0,449
Decision Tree	`maxDepth` = 3	0,434	0,427

Le Random Forest (0,451) et la Logistic Regression (0,449) sont **quasiment à égalité**. On applique la règle fixée à l'avance : le modèle retenu est celui qui a la meilleure accuracy de validation, le **Random Forest**.

6.3 Étape 3 : évaluation finale sur le test set

Tab. 6.3. Performances sur le test set (1 660 individus de ménages jamais vus)

Modèle	Accuracy	Weighted F1	Recall		
			Peu	Mobile	Très
Baseline (classe majoritaire)	0,361	0,192	0,000	1,000	0,000
Random Forest (`maxDepth` = 8, `minInstancesPerNode` = 5)	**0,447**	**0,440**	0,520	0,500	0,258

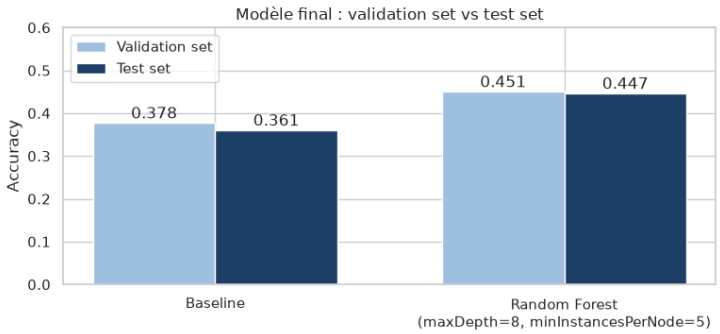


Fig. 6.3. Accuracy du modèle final et de la baseline sur le validation set et sur le test set

Le modèle final gagne **8,6 points** d'accuracy sur la baseline. Son accuracy de test (0,447) est très proche de celle de validation (0,451) : le score annoncé est **fiable**, le modèle ne s'est pas sur-adapté au validation set pendant les choix.

6.4 Confusion matrix

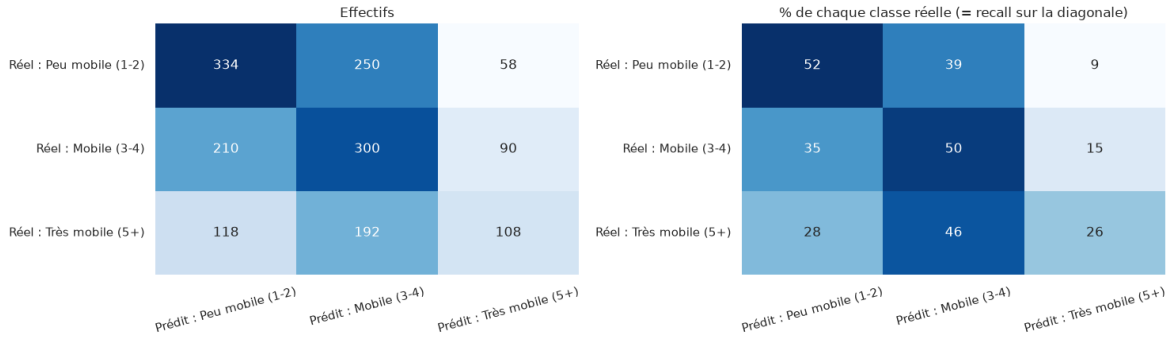


Fig. 6.4. Confusion matrix du modèle final sur le test set (effectifs et pourcentages par classe réelle)

La confusion matrix (figure 6.4) montre que :

- les classes *peu mobile* (52 %) et *mobile* (50 %) sont reconnues environ une fois sur deux ;
- la classe *très mobile* est la plus difficile (26 %) : 46 % de ses membres sont prédits *mobiles*, car rien dans le profil ne distingue une personne faisant 6 déplacements d'une personne en faisant 4 ;
- les erreurs portent surtout sur des **classes voisines**.

6.5 Faux positifs et faux négatifs

Avec trois classes, les faux positifs (FP) et faux négatifs (FN) sont calculés pour chaque classe, en la comparant aux deux autres (*one-vs-rest*). Pour une classe donnée, un FN est une personne de cette classe que le modèle a classée ailleurs (reste de la ligne dans la confusion matrix), et un FP une personne classée à tort dans cette classe (reste de la colonne).

Tab. 6.4. TP, FP, FN, precision et recall par classe (test set)

Classe	TP	FP	FN	TN	Precision	Recall
Peu mobile	334	328	308	690	0,505	0,520
Mobile	300	442	300	618	0,404	0,500
Très mobile	108	148	310	1 094	0,422	0,258

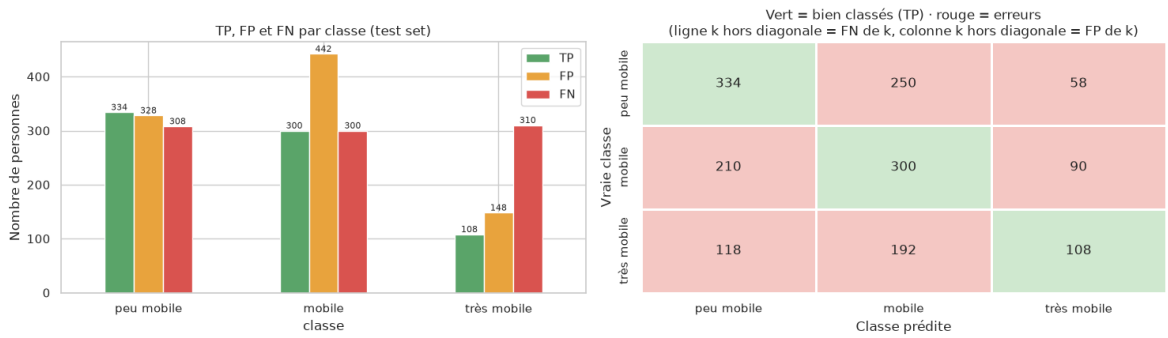


Fig. 6.5. TP, FP et FN par classe, et confusion matrix colorée (vert : bien classés, rouge : erreurs)

Lecture

- **Très mobile** : beaucoup de FN (310) pour peu de TP (108). Le modèle **rate 3 très mobiles sur 4**, mais quand il prédit « très mobile », il a raison dans 42 % des cas : il est prudent et prédit rarement cette classe.
- **Mobile** : le plus de FP (442). C'est la classe « du milieu » : quand le modèle hésite, il répond souvent « mobile ».
- **Peu mobile** : la classe la plus équilibrée (precision 0,51, recall 0,52). Le modèle est donc plus utile pour repérer les personnes peu mobiles que les très mobiles.

6.6 Feature importance

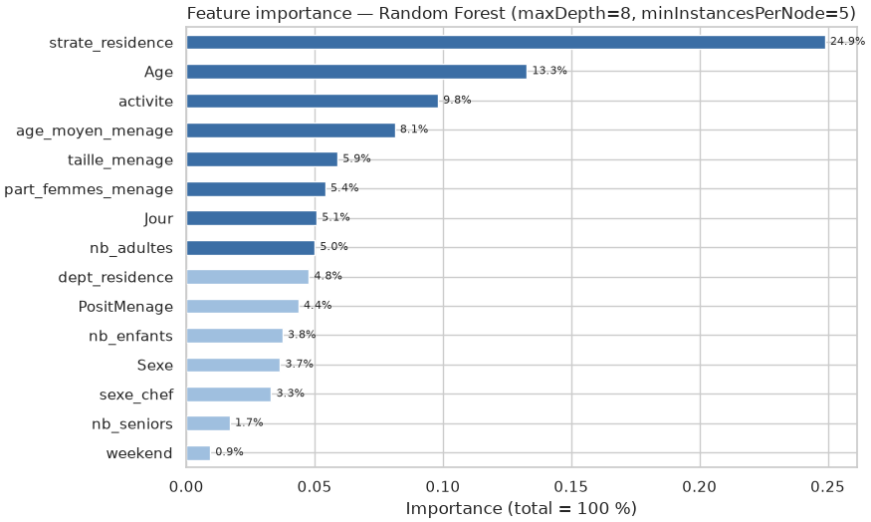


Fig. 6.6. Feature importance du Random Forest retenu

La **zone de résidence** (25 %) arrive en tête, suivie de l'**âge** (13 %), de l'**activité** (10 %) et de l'**âge moyen du ménage** (8 %). Le jour et le week-end comptent très peu,

ce qui confirme l'EDA. La feature importance de la strate de résidence est toutefois surestimée : les Random Forests favorisent les variables catégorielles à nombreuses modalités.

6.7 Analyse des erreurs

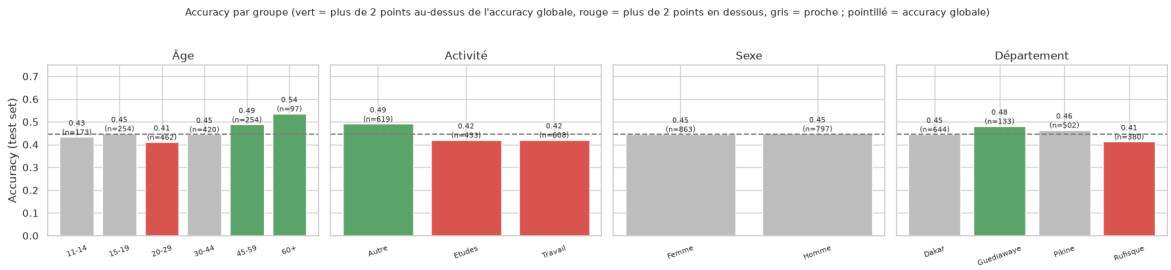


Fig. 6.7. Accuracy du modèle final selon le profil (test set)

Le modèle prédit mieux les **plus de 45 ans** (0,49 à 0,54) et les personnes **sans travail ni études** ce jour-là (0,49), dont le comportement est plus régulier. Il se trompe davantage pour les **20–29 ans** (0,41), les **élèves et les actifs** (0,42) et les habitants de **Rufisque** (0,41), dont la mobilité dépend d'éléments non observés (type d'emploi, horaires, distances). Il n'y a **aucune différence selon le sexe** (0,45 pour les hommes comme pour les femmes).

6.8 Confiance du modèle

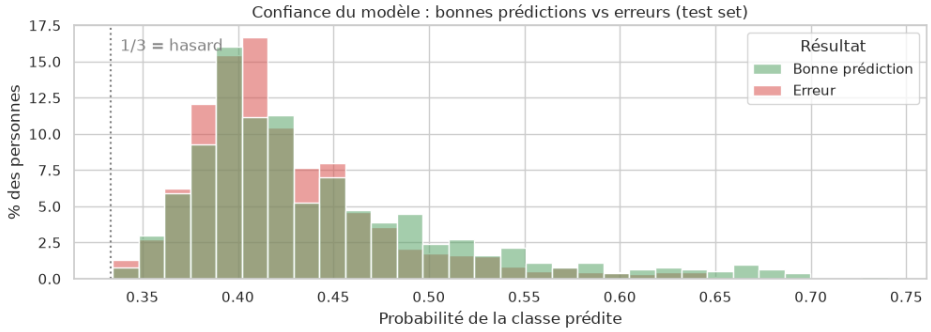


Fig. 6.8. Probabilité de la classe prédite pour les bonnes prédictions et pour les erreurs

Les probabilités de la classe prédite sont presque toutes comprises entre 0,35 et 0,50, à peine au-dessus du hasard (1/3) : le modèle est **rarement sûr de lui**. Les distributions des bonnes prédictions et des erreurs se recouvrent presque entièrement (médianes 0,42 et 0,41) ; ce n'est qu'au-delà de 0,55 que les bonnes prédictions dominent.

6.9 Exemple d'utilisation du modèle

Le modèle final est appliqué à des profils fictifs, créés directement en Spark SQL (tableau 6.5 et figure 6.9).

Tab. 6.5. Prédictions pour quatre profils fictifs

Âge	Sexe	Activité	Résidence	Niveau prédit	Probabilités (peu / mobile / très)
14	Homme	Études	Guédiawaye	Très mobile	0,15 / 0,33 / **0,52**
35	Femme	Autre	Dakar	Peu mobile	**0,42** / 0,41 / 0,17
40	Homme	Travail	Pikine	Peu mobile	**0,39** / 0,36 / 0,25
67	Femme	Autre	Rufisque	Peu mobile	**0,55** / 0,32 / 0,13

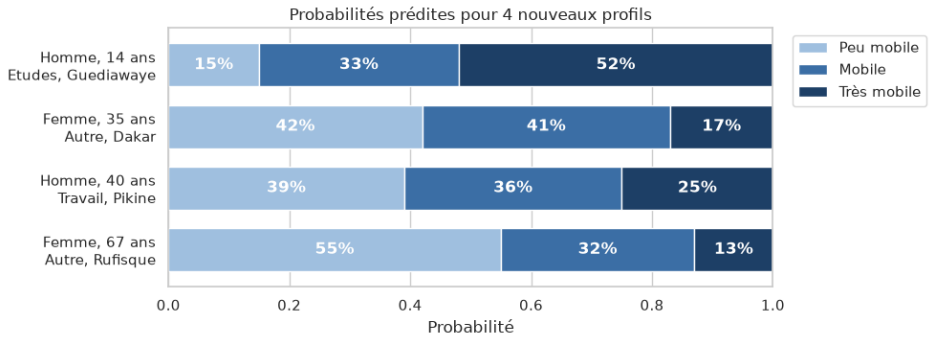


Fig. 6.9. Probabilités prédites pour quatre nouveaux profils

Les profils typés sont nettement classés (l'élève de 14 ans est « très mobile », la femme de 67 ans « peu mobile »), alors que le modèle **hésite** pour les profils « moyens » (42 % contre 41 % pour la femme de 35 ans), exactement ceux où il se trompe le plus.

6.10 Synthèse du chapitre

Le modèle retenu, un Random Forest, atteint 44,7 % d'accuracy sur le test set contre 36,1 % pour la baseline, un score proche de celui de validation. Les courbes de réglage montrent l'overfitting du Decision Tree et la robustesse du Random Forest. Les erreurs portent surtout sur des classes voisines, et la classe *très mobile*, avec beaucoup de faux négatifs, est la plus difficile à reconnaître.

Chapitre 7

Discussion

7.1 Interprétation

Les résultats confirment les grandes tendances connues de la mobilité dans les villes d'Afrique de l'Ouest :

- une **mobilité scolaire** très intense, avec des allers-retours multiples dans la journée ;
- une **inégalité de genre** : les femmes adultes, et en particulier celles sans activité de travail ou d'études, se déplacent moins ;
- une **mobilité plus élevée en banlieue** (Rufisque, Guédiawaye) qu'à Dakar ville, où les équipements sont plus proches.

La performance modeste du modèle est elle-même un résultat : **le profil socio-démographique n'explique qu'une partie de la mobilité individuelle**. Deux personnes au profil identique peuvent avoir des journées très différentes. Le fait que le Random Forest et la Logistic Regression atteignent presque le même score, et que le réglage des hyperparameters ne change presque rien, confirme que la limite vient **des données et non de l'algorithme**.

7.2 Limites

1. **Variables manquantes** : le revenu, la profession, la possession d'un véhicule et la distance aux lieux d'activité, déterminants connus de la mobilité, ne figurent pas dans les données. C'est la principale limite.
2. **Absence d'individus immobiles** : la target ne couvre pas le cas « 0 déplacement ».
3. **Activité déduite** : dans le scénario B, l'activité est tirée des motifs de la journée et non déclarée ; elle doit être vue comme un indicateur de statut.
4. **Une seule journée observée** par individu : la variabilité d'un jour à l'autre n'est pas mesurée.

7.3 Bonnes pratiques Big Data mises en œuvre

Tab. 7.1. Bonnes pratiques Big Data appliquées

Pratique	**Mise en œuvre**
Schéma explicite	33 champs typés, aucun `inferSchema` sur la table principale
Mise en cache	`cache()` des tables réutilisées (nettoyée, individus, train, validation et test sets)
Agrégation distribuée	`groupBy` déplacements → individus, `max_by`
Fonctions de fenêtre	Variables de ménage sans jointure ni collecte
Collecte minimale	Seuls les petits résultats agrégés sont convertis en pandas
Pipelines MLlib	Encodage et modèle dans un même `Pipeline` reproductible
Sélection de modèle	Choix du scénario, du modèle et des hyperparameters sur le validation set ; test set utilisé une seule fois
Prévention du data leakage	Split train / validation / test par ménage, exclusion des variables issues de la journée
Réutilisation du modèle	`PipelineModel` complet appliqué à de nouveaux profils

Conclusion générale

Ce projet a permis de construire une chaîne complète de traitement Big Data, de l'EDA à la modélisation distribuée, sur l'enquête ménages-déplacements de Dakar. L'EDA a guidé chacune des décisions de préparation : correction des heures et des codes, choix de la zone de résidence, définition de la target et exclusion des variables sources de data leakage.

Sur le plan thématique, l'étude montre qu'un Dakarois mobile réalise en moyenne **3,9 déplacements par jour**, que les jeunes, les élèves, les hommes et les habitants de la banlieue sont les plus mobiles, et que les journées s'organisent en allers-retours depuis le domicile. Sur le plan prédictif, après un découpage train / validation / test (70 / 15 / 15) par ménage, le modèle retenu (Random Forest) atteint une accuracy de **44,7 %** sur le test set, contre 36,1 % pour la baseline : le profil d'une personne renseigne sur sa mobilité, mais n'en explique qu'une partie. La classe des personnes très mobiles reste la plus difficile à repérer.

Perspectives.

- Modéliser directement le nombre de déplacements par une **régression de Poisson** (`GeneralizedLinearRegression`).
- Réduire la target à **deux classes** (peu mobile / mobile), une tâche plus facile.
- Prédire le **type de journée** (chaîne domicile → travail → domicile, domicile → école → marché → domicile, etc.).
- Enrichir les données par des sources externes (recensement, revenu, motorisation des ménages, distances réelles) afin de lever la principale limite identifiée.
- Déployer le pipeline sur un cluster Spark pour traiter plusieurs vagues d'enquête ou des données de traces numériques.

Annexe A

Dictionnaire des variables construites

Tab. A.1. Variables créées au niveau individu

Variable	Définition
`nb_dep`	Nombre de déplacements de l'individu dans la journée
`niveau`	Cible : 0 (1–2 déplacements), 1 (3–4), 2 (5 et plus)
`strate_residence`	Strate d'arrivée du dernier déplacement de la journée
`dept_residence`	Département déduit de la strate (Dakar, Pikine, Guédiawaye, Rufisque, Extérieur, Inconnu)
`activite`	Travail si un motif de travail, sinon Études si un motif d'études, sinon Autre
`taille_menage`	Nombre de personnes mobiles du ménage
`nb_enfants`, `nb_adultes`, `nb_seniors`	Membres de moins de 18 ans, de 18 à 59 ans, de 60 ans et plus
`age_moyen_menage`	Âge moyen des membres du ménage
`part_femmes_menage`	Proportion de femmes dans le ménage
`sexe_chef`	Sexe du chef de ménage
`weekend`	1 si l'enquête porte sur un samedi ou un dimanche
`poids`	Poids d'extrapolation de l'individu

Annexe B

Règle de calcul du département

Extrait de code B.1. Département à partir du code de strate, codes spéciaux inclus

```
dept_map = F.create_map([F.lit(x) for x in
    ["D", "Dakar", "P", "Pikine", "G", "Guediawaye", "R", "Rufisque"]])

def departement(col):
    return (F.when(col == "zex", "Exterieur")              # hors zone d'étude
              .when(col == "NA_region", "Inconnu")          # localisation inconnue
              .when(col.startswith("NA"),                   # NA_d, NA_p, NA_g, NA1_r
                    dept_map[F.upper(F.substring_index(col, "_", -1))])
              .otherwise(dept_map[F.substring(col, 1, 1)]))  # D05 -> Dakar
```

Le code complet est fourni dans le notebook `Projet_BigData_Niveau_Mobilite_Dakar.ipynb`.